Assignment 6
Handling Temporal Coupling

Introduction to Functional Programming | Academic year 2026 2027

Maximum score 15 points | Individual work or teams of 2 to 4 students

Refactor a project workflow so that correctness no longer depends on remembering a hidden order of mutating method calls. Use validated factories, explicit states, and pattern matching to make prerequisites visible and invalid operations safe.

Use your planned final project or a reusable module that will be integrated into it. Keep one runnable project across Assignments 5 to 8. A console interface is sufficient; a new application or a graphical interface is not required for each assignment.

Every team member must implement meaningful logic, contribute automated tests, and defend their own work. Agree on responsibilities before coding. Earlier relevant code and tests may be reused and improved.

Task 1 Identify the hidden dependency

Choose an operation from your application, such as confirming a booking, registering an eligible student, or calculating a quote with validated rules. Show a small unsafe before example in README.md. Identify the prerequisite, the required order, and what goes wrong when the order is changed or a step is omitted.

For example, Confirm() may accidentally succeed before Validate() has run, or Calculate() may use an unset configuration because LoadRules() was forgotten. Explain the observable problem, such as an invalid confirmation or an incorrect amount. Keep the unsafe example in documentation or a separate demonstration; it must not remain the active application path.

Task 2 Create validated values with a factory

Implement and use a factory that creates a validated domain value, workflow state, or configured policy. The factory must check the actual project rules before it returns a usable value. Include at least two meaningful checks, such as a positive quantity and an existing category, or a valid date range and sufficient capacity.

Ensure external callers cannot freely bypass the checks with an unrestricted constructor or mutable property. Explain the chosen access restrictions. The factory must either return the valid value or explicitly report why construction failed; partial construction must not escape as a valid state.

Use the created value in the real application workflow. A factory that is implemented only for a test and never used by the application does not satisfy the task. You may preserve your existing validation approach and improve its error model in Assignment 7.

Explicit states and safe transitions

Task 3 Model distinct cases

Create a discriminated union style model with at least two mutually exclusive cases. C# implementations may use a closed record hierarchy or an explicit tagged model whose valid combinations are enforced. Each case must carry the data relevant to that case; a single boolean flag with freely editable unrelated fields is insufficient.

Useful examples include a validation outcome with Valid carrying a validated value and Invalid carrying reasons, or a workflow with Pending and Confirmed carrying different data. State which cases your model allows and what each case means.

Task 4 Handle the cases with pattern matching

Use a switch expression or pattern matching in an actual domain path. Handle every case defined by your model explicitly. A fallback may reject an unexpected case, but must not silently treat it as a valid one. Do not use casts or null-forgiving access to assume a payload exists for the wrong case.

The model and matching may be part of the factory outcome or of the workflow transition. They must influence the application result; a disconnected pattern matching demonstration does not count.

Task 5 Make operations safe

Replace the hidden sequence with explicit inputs or states. A transition can accept a validated value rather than a raw request, or explicitly reject a state whose prerequisites are not met. Legitimate domain order may remain, but callers must be able to see the requirement in the API or receive a clear rejection.

Keep decisions and calculations pure where possible. Return a new value or next state rather than mutating shared state during validation. Validate all prerequisites before committing any update at the boundary. A failure must leave the original object, collection, and stored data consistent.

Demonstrate one successful operation and at least two different invalid cases. One invalid case must involve a missing prerequisite or an invalid sequence. Define what happens when an operation is repeated: reject it, return the existing outcome, or apply a documented safe rule.

Example of an acceptable design

A booking factory validates the request and returns a Valid or Invalid case. The Valid case carries a ValidatedBooking that external callers cannot construct unchecked. Confirmation accepts that validated value or checks the explicit current state. Pattern matching reports validation reasons or proceeds to confirmation. A repeated confirmation follows a documented rule. This illustrates visible prerequisites; adapt the models and transitions to your project.

Tests and individual defense

Provide at least 8 cumulative automated tests. Use an assertions-based test framework or a test runner that compares actual and expected values and reports failures. Console output alone is not a test. Relevant earlier tests count toward the minimum.

The cases below define required coverage. One test may contain several assertions, and one scenario may cover several requirements. Use additional tests if needed; the minimum number does not remove any required case.

CASE	REQUIRED ASSERTION
FACTORY SUCCESS	Valid input creates a usable validated value with the expected data.
FACTORY REJECTION	Each required validation rule rejects invalid input and reports the reason.
DISTINCT CASES	Construct reachable examples of every declared case and check the corresponding pattern matching result.
NORMAL OPERATION	The validated value is used by the real workflow and produces the correct next state or outcome.
MISSING PREREQUISITE	An invalid sequence is rejected without producing an invalid success.
REPEATED OPERATION	A second call follows the documented safe behaviour.
BOUNDARY VALUE	An exact limit follows the factory or transition rule.
STATE PRESERVATION	Assert that a failed construction or transition leaves original state unchanged.

If an invalid call is prevented at compile time, describe that call in README.md rather than adding uncompilable code to the submitted project. Test the relevant runtime validation and successful transition.

What to show during the defense

- Compare the unsafe and safe call sites and explain the hidden dependency.
- Show where the factory enforces rules and what prevents unchecked construction.
- Demonstrate each case and explain the payload and pattern matching branch.
- Run a valid operation, two invalid cases, and the repeated-operation scenario.
- Modify a validation rule or transition and verify the change with an automated test.

Keep the safe workflow in the project for Assignment 7. Its valid and invalid outcomes may become your Result model or remain separate domain states, depending on the meaning of each type.

Assessment and submission

Assessment

CRITERION	EVIDENCE ASSESSED	POINTS
WORKFLOW CORRECTNESS	Valid operations work; failures and repeated operations preserve consistent state.	1
EXPLICIT DEPENDENCIES	Inputs or states expose prerequisites; distinct cases are handled through used pattern matching.	2
VALIDATED CONSTRUCTION	A used factory enforces domain rules and prevents unchecked construction.	2
FUNCTIONAL SEPARATION	Domain decisions are isolated from input and output and avoid partial updates.	1
TESTS AND DOCUMENTATION	At least 8 relevant cumulative tests, case coverage, and a clear before and after example.	2
INDIVIDUAL DEFENSE	Explain your factory and states and complete a small live modification.	7
TOTAL		**15**

The implementation criteria total 8 points; the individual defense is worth 7 points. Each student receives an individual grade. Only implemented constructs used by the application earn points. A student who does not defend receives 0. If you cannot explain your solution, it is not accepted under the course code defense policy. Academic integrity and AI rules follow the syllabus.

What to upload

Every student must upload a ZIP archive with a runnable snapshot for Assignment 6. Team members may upload the same project code, but each must identify and defend their own contribution. Use the deadline and submission policy published in Moodle.

- Include C# source, project and solution files, automated tests, README.md, and sample input files needed to run the demonstrated workflows. Exclude generated .NET bin and obj folders, IDE caches, and local secrets.
- State the .NET SDK version, exact build, run and test commands, expected working directory, and any prerequisites. Check these instructions from a clean extraction of the archive.

README content

- Describe the project workflow, its input, domain rules, expected output, and short instructions for the normal and failure demonstrations.
- Map each requirement to the files and methods that implement it. List test cases, expected results, and the latest test outcome.
- List each member by name and group and explain their implemented logic, tests, and documentation for this stage. Team size does not increase the number of required workflows or reports.

Use in the final project

Retain the code and tests for the next stage. The final project must be hosted in an accessible Git repository on GitHub, GitLab, or another Git hosting service. Repository links do not replace the files required in Moodle. The final project has a separate assessment of 50 points for the project and 50 points for the individual defense, with a group project report and an individual contribution report.